

# Appendix A - Stage 3: vectorizing the cache-resident stencil

Matan Cohen (ID: 208304410) · Yuval Kogan (ID: 310596424)

HW/SW Co-Design - HW1 · supplement to the main report (kept separate to respect the 3-page limit)

Code & data: [github.com/matanco64/HWSW\\_HW1\\_CBRAM](https://github.com/matanco64/HWSW_HW1_CBRAM)

The main report ends on an open question: Stage 2 (time-skewing) made the Jacobi sweep **cache-resident** - the best memory behaviour of the three layouts - but lost 16% to SoA because the halo recomputation **added** 40% instructions, and on a single core the traffic it saved was no longer the limiter. That diagnosis names its own cure: if the loss is **added compute**, make the compute cheap. Stage 3 does exactly that, and it is the fastest stage in the project - **83.53** s, **4.5×** over the baseline, **2.94×** over SoA, and **3.41×** over the Stage 2 it builds on - while staying bit-identical to every other stage.

## A The lever: the stencil was never vectorized

Inspecting GCC's vectorizer report (-fopt-info-vec-missed) on Stages 1 and 2 revealed that **the 4-point Jacobi stencil - the hot loop - ran scalar in both**. GCC 15.2 (-O2 -march=native) bailed with “complicated access pattern” / “not profitable”: the grid is reached through `std::vector::operator[]`, so the compiler cannot prove the read buffer `cur` and the write buffer `nxt` (or the `metal` array) do not alias, and its cost model then vetoes the transform. The Xeon E5-2630 v3 (Haswell) has AVX2 - **8 int32 lanes per 256-bit vector** - so the stencil was leaving ~ 8× of integer throughput on the table. Crucially this matters **most** for Stage 2: its tile is L2-resident, so it is **throughput-bound**, and vectorized arithmetic converts almost directly to wall-clock. The same SIMD applied to Stage 1 would still stall on DRAM latency and help far less. **Vectorization is the missing complement to time-skewing** - cache-residency and compute throughput exploit each other.

## B The change: three edits, all inside `advance_tile()`

`dbm_stage3.cpp` is `dbm_stage2.cpp` with three localized edits; everything else (`block_advance`, `solve_potential`, candidate selection, PRNG, IO, all constants) is copied verbatim, so the **only** variable is the stencil codegen.

1. **Vectorize the stencil.** Hoist `__restrict` raw pointers and annotate the inner run with `#raw("#pragma omp simd")` (compiled with `-fopenmp-simd`, no OpenMP runtime). This removes the alias ambiguity **and** overrides the cost-model veto. GCC then reports **loop vectorized using 32 byte vectors**. The arithmetic shift `>> 2` maps to `vpsrad(_mm256_srai_epi32)`, exact on these non-negative averages.
2. **Fuse the Dirichlet re-pin into the store.** Stage 2 re-pinned metal cells to  $V = 0$  in a **separate pass** over the tile every sweep. Folding it into the stencil store as a branchless select writes each cell once and deletes an entire per-sweep pass:
3. **`__restrict` the tile copy-in/out** so they emit `memcpy`-style vector moves.

```
const int32_t* __restrict cv = cur->data(); // hoisted, non-aliasing
int32_t* __restrict nv = nxt->data();
const uint8_t* __restrict mv = m.data();
#pragma omp simd
for (int c = cc0; c < cc1; ++c) {
    size_t li = base + c;
    int32_t avg = (cv[li - LW] + cv[li + LW] + cv[li - 1] + cv[li + 1]) >> 2;
    nv[li] = mv[li] ? 0 : avg; // re-pin fused into the store
}
```

## C Result: the instruction count collapses

| Stage                                 | Time (s)     | vs base     | Instr (B)  | IPC   | L1-miss % |
|---------------------------------------|--------------|-------------|------------|-------|-----------|
| Stage 0 - AoS                         | 375.82       | 1.00×       | 1508       | 1.746 | 19        |
| Stage 1 - SoA                         | 245.88       | 1.53×       | 1343       | 2.359 | 7.3       |
| Stage 2 - SoA + skewing               | 284.92       | 1.32×       | 1881       | 2.845 | 4.25      |
| <b>Stage 3 - SoA + skewing + SIMD</b> | <b>83.53</b> | <b>4.5×</b> | <b>408</b> | 2.11  | 15.24     |

The mechanism is visible directly in the counters: instructions retired fall **1881 B → 408 B** (a **4.61×** cut) - the roughly 8×{ } narrower stencil arithmetic plus the deleted re-pin pass - and cycles drop **661 B → 193 B** (3.42×). IPC **decreases** (2.845 → 2.11), which is exactly right: each vector instruction now does the work of eight scalar ones, so there are far fewer, heavier instructions in flight

- fewer instructions at slightly lower IPC still finishes in **3.42×**

the cycles. L1-miss rises (4.25% → 15.24%) because the vector loads pull more bytes per instruction, but the tile is L2-resident so those misses are cheap and the throughput gain dominates. This is the payoff the main report predicted: time-skewing's cache-residency was latent headroom, and vectorizing its now-resident stencil **cashes it in** - the slowest “optimization” becomes the fastest stage.

## D Correctness preserved

Stage 3 clears the same gate as every other stage: `V_final_stage3.bin` is **bit-identical** to Stage 0's (`cmp` silent, `exit 0`; `run.sh` enforces this before trusting any perf number). This holds because the fixed-point `int32` add + arithmetic shift is exact and order-independent - `_mm256_srai_epi32` matches scalar `>> 2` lane-for-lane - and the fused re-pin writes the identical final value. Vectorizing the math changed **how** the bytes are produced, not **which** bytes, so the SIMD kernel is provably equivalent, not merely close.

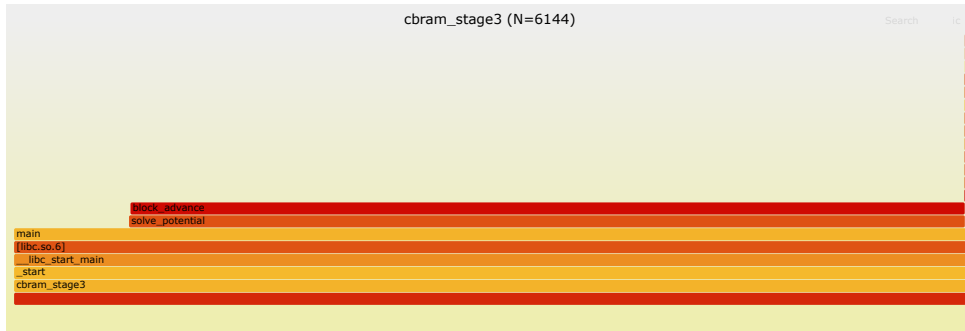


Figure 1: Stage 3 self-time flame graph (`perf record -F 999, N = 6144`). The `advance_tile` stencil that dominated Stage 2 is now a thin, vectorized band - the runtime is no longer spent in scalar stencil arithmetic.

**Next step.** Stage 3 is single-core. The remaining headroom is the **other** path the main report names: parallelizing tiles across cores so the shared memory bus becomes the binding resource again - at which point Stage 2's traffic savings, dormant on one core, would finally pay off too. That multicore stage is the natural follow-up.